

DOCUMENTAÇÃO DDD-JAVA – CHALLENGE TOTVS 2026

1. Descrição do Projeto

Nome da Solução

TOTVS Meeting Intelligence

Objetivo

O projeto tem como objetivo desenvolver uma solução capaz de processar transcrições de reuniões comerciais e gerar inteligência de negócio de forma automatizada.

A aplicação recebe uma transcrição textual de uma reunião entre um cliente e representantes da empresa. A partir dessa transcrição, o sistema identifica informações relevantes como oportunidades comerciais, riscos de cancelamento, produtos mencionados, sentimentos demonstrados pelo cliente e possíveis ações futuras.

A solução foi inspirada no desafio proposto pela TOTVS, que busca transformar conversas não estruturadas em informações acionáveis para auxiliar equipes comerciais e gestores na tomada de decisão.

O sistema foi modelado seguindo conceitos de Orientação a Objetos e Domain Driven Design (DDD), permitindo que a estrutura do software seja compatível com o modelo de dados desenvolvido na disciplina de Database Design.

2. Principais Funcionalidades

Funcionalidade 1 – Registrar Reunião

Permite cadastrar uma nova reunião realizada com um cliente, armazenando informações como data, responsável e descrição.

Funcionalidade 2 – Registrar Transcrição

Permite associar uma transcrição textual à reunião cadastrada.

Funcionalidade 3 – Executar Análise

Permite gerar uma análise da transcrição, produzindo indicadores de risco, oportunidade e sentimento.

Funcionalidade 4 – Gerar Insights

A partir da análise realizada, o sistema registra insights que podem representar oportunidades comerciais, riscos ou observações importantes.

Funcionalidade 5 – Consultar Histórico

Permite consultar todas as reuniões e análises associadas a um determinado cliente.

3. Modelagem UML

Classe Cliente

Atributos:

- idCliente : int
- nome : String
- segmento : String
- porte : String

Métodos:

- exibirDados()
 - atualizarCadastro()
-

Classe Reuniao

Atributos:

- idReuniao : int
- data : String
- titulo : String
- responsavel : String

Métodos:

- cadastrarReuniao()
 - exibirReuniao()
-

Classe Transcricao

Atributos:

- idTranscricao : int
- textoBruto : String
- idioma : String

Métodos:

- cadastrarTranscricao()
 - exibirTranscricao()
-

Classe ProdutoTotvs

Atributos:

- idProduto : int
- nomeProduto : String
- categoria : String

Métodos:

- exibirProduto()
-

Classe AnaliseIA

Atributos:

- idAnalise : int
- sentimentoGeral : String
- scoreRisco : double
- scoreOportunidade : double

Métodos:

- gerarAnalise()
 - exibirResultado()
-

Classe Insight

Atributos:

- idInsight : int
- tipoInsight : String
- descricao : String
- prioridade : String

Métodos:

- gerarInsight()
 - exibirInsight()
-

4. Relacionamento Entre Classes

Cliente → Reuniao

Um cliente pode possuir várias reuniões.

Reuniao → Transcricao

Uma reunião possui uma transcrição.

Reuniao → AnaliseIA

Uma reunião pode gerar uma ou mais análises.

AnaliseIA → Insight

Uma análise pode gerar diversos insights.

ProdutoTotvs → Insight

Um insight pode estar associado a um produto TOTVS identificado na conversa.

5. Estrutura de Pacotes

model

- Cliente
- Reuniao
- Transcricao
- ProdutoTotvs
- AnaliseIA
- Insight

service

- AnaliseService
- InsightService

view

- Main
-

6. Recursos de Orientação a Objetos

Encapsulamento

Todos os atributos serão privados e acessados por getters e setters.

Herança

Será criada uma classe abstrata chamada Pessoa.

Classes filhas:

- Cliente

- UsuarioSistema

Polimorfismo

Método `exibirDados()` poderá ser sobrescrito pelas classes derivadas.

Associação

As classes possuem referências entre si para representar os relacionamentos do negócio.

7. Classe Principal

A classe Main deverá:

1. Instanciar um cliente.
2. Criar uma reunião.
3. Registrar uma transcrição.
4. Executar uma análise.
5. Gerar insights.
6. Exibir os resultados na tela utilizando `Scanner` ou `JOptionPane`.

Dessa forma será possível validar todas as classes e funcionalidades exigidas pelo desafio.